

ARGOS SLOPE 4.0 — Informe Técnico de Avance

Análisis basado en revisión directa del código fuente

Fecha: Junio 2026 | Revisado por: Antigravity AI | Base: e:\PROYECTO-QUICKSTART\ProyectoMineria\ArgosSlope4.0

1. Resumen Ejecutivo

ARGOS SLOPE 4.0 es un sistema distribuido de monitoreo geotécnico en tiempo real para taludes mineros. Su meta es construir un **gemelo digital 3D métrico** que integre captura visual en campo, detección de fisuras con clasificación, medición calibrada, historial evolutivo y visualización 3D interactiva.

Tras la revisión directa del código fuente, se confirma que el proyecto tiene una **base técnica sólida y en ejecución**. El módulo Edge en Python (`edge/edge/`) está operativo con un pipeline completo que incluye detección de fisuras por OpenCV, estimación de profundidad con MiDaS (IA monocular), generación de nubes de puntos y transmisión de video en vivo por WebRTC y MJPEG. El Backend (.NET) expone una API REST con modelos de datos relacionales bien definidos. El Frontend (Next.js) renderiza en tiempo real la nube de puntos 3D usando React Three Fiber, y muestra los bounding boxes de fisuras sobre el video vía MQTT WebSocket.

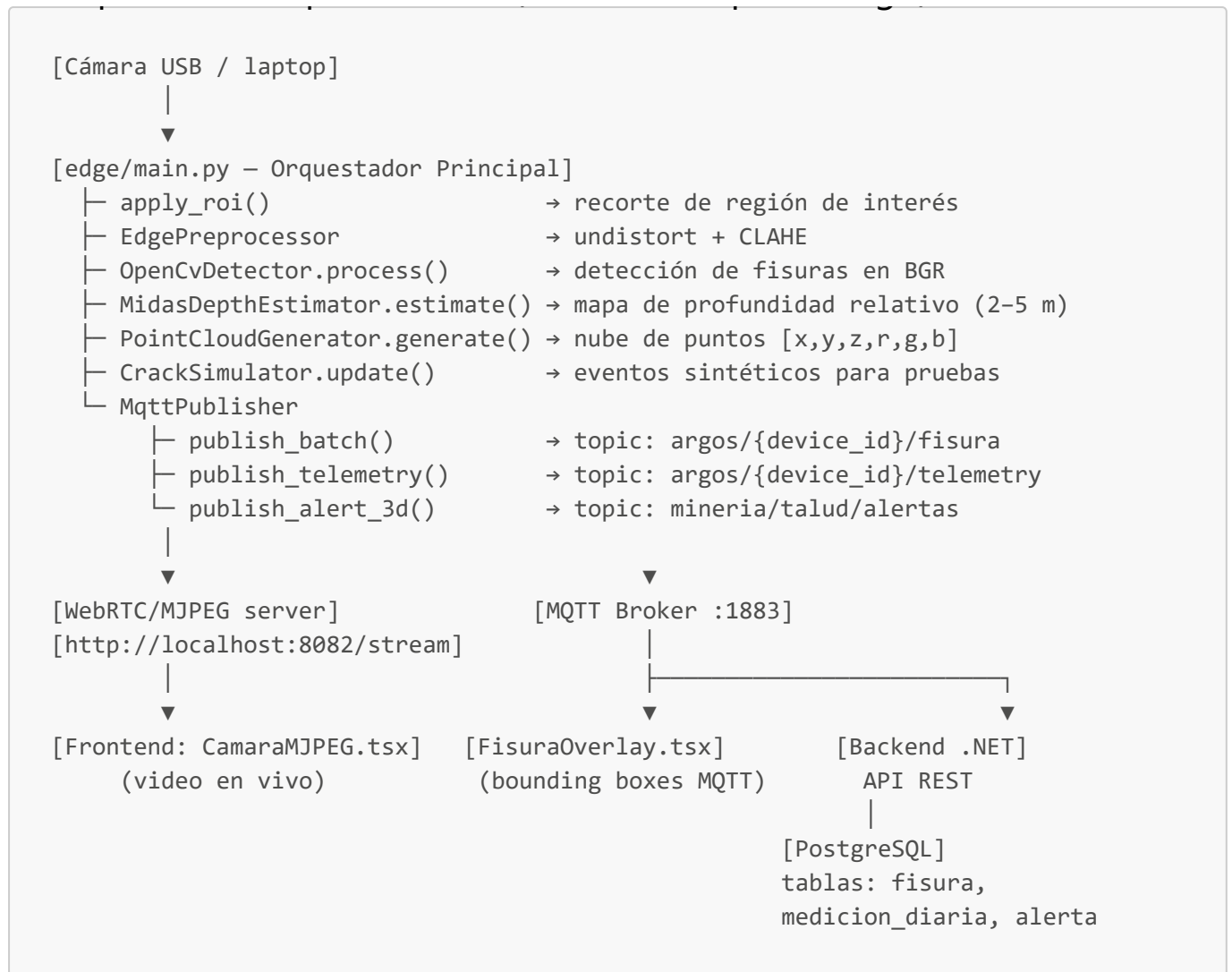
La principal brecha confirmada por código es la **calibración no ejecutada**: el archivo `calibration/calibration.json` tiene todos sus campos en `null`. Esto significa que las mediciones actuales de longitud y ancho en milímetros se calculan con parámetros físicos estimados por defecto y **no son confiables para uso en minería real**.

2. Meta Final del Sistema

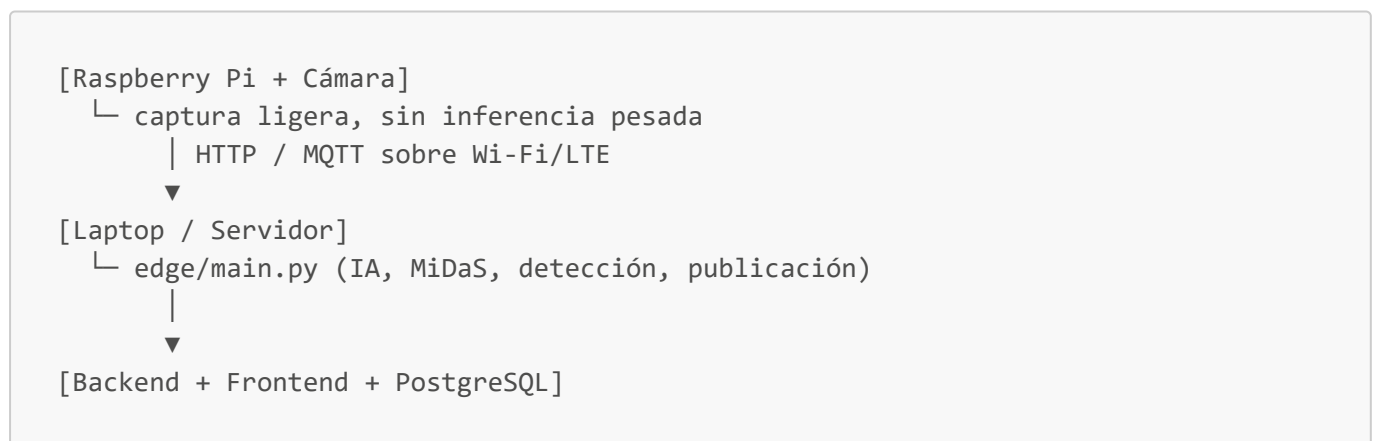
El sistema busca ser un **Gemelo Digital 3D para monitoreo de taludes**, no un simple dashboard. Esto implica:

Capacidad	Descripción técnica
Captura visual continua	Cámara USB o Raspberry Pi → OpenCV
Detección de fisuras	OpenCV (activo) o modelo ONNX/YOLO (preparado)
Medición calibrada	<code>length_mm</code> , <code>width_mm</code> , <code>area_mm2</code> via factor <code>px/mm</code>
Comparación temporal	Evolución de <code>DeltaPorcentaje</code> por fisura en BD
Risk Engine	Clasificación: <code>fina</code> / <code>media</code> / <code>gruesa</code> → alertas
Historial y trazabilidad	PostgreSQL con tablas <code>fisura</code> , <code>medicion_diaria</code> , <code>alerta</code>
Visualización 3D	React Three Fiber + MiDaS + nube de puntos vía MQTT
Visualización inmersiva (futura)	WebXR / VR sobre el gemelo digital

3. Arquitectura Implementada (Evidenciada por Código)



Arquitectura Futura (Hardware en campo):



4. Análisis Detallado por Módulo

4.1 Módulo Edge — Orquestador Principal

 **edge/edge/main.py** (506 líneas)

Este es el **motor central del sistema**. Gestiona concurrentemente todos los subsistemas.

Flujo por frame:

1. `cap.read()` — captura BGR a 1280×720 @ 15 FPS (configurables por `.env`)
2. `apply_roi()` — recorta zona de interés (`x,y,w,h` por variable de entorno)
3. `EdgePreprocessor.process()` — aplica `undistort` (si hay calibración) y `CLAHE`
4. `detector.process()` — detecta fisuras y devuelve lista de `CrackResult`
5. Si `depth_enabled`: `MiDaS.estimate()` + `PointCloudGenerator.generate()` cada N frames
6. Si `simulator_enabled`: `CrackSimulator.update()` genera eventos sintéticos
7. `publisher.publish_batch()` — publica cada fisura por MQTT (QoS 1)
8. `publisher.publish_alert_3d()` — publica nube de puntos + fisuras 3D
9. `shared_frame.write()` — alimenta el servidor WebRTC/MJPEG en tiempo real
10. Cada `telemetry_interval_s`: publica FPS, temperatura CPU, conteo de fisuras

Configuración total por variables de entorno: La clase `EdgeConfig` (218 líneas) cubre 40+ parámetros sin modificar código: cámara, detector, MQTT, thresholds, WebRTC, profundidad, filtros geométricos, calibración.

Estado real: ☒ **Completamente operativo en laptop.** Compatible con Raspberry Pi (detección de `/etc/hostname` para `device_id`, lectura de temperatura de sysfs).

4.2 Detector de Fisuras

 `edge/edge/detector/fisura_detector.py` (603 líneas)

Es el módulo de mayor impacto técnico. Implementa **dos backends intercambiables**:

Pipeline OpenCV (`OpenCvDetector`)

```
Imagen BGR
→ Escala de grises (cv2.cvtColor)
→ Blur Gaussiano (kernel configurable, default: 5×5)
→ Umbral adaptativo Gaussiano THRESH_BINARY_INV
  block_size: 31 | C: 5 (configurables)
→ Operaciones morfológicas: CLOSE → OPEN (elimina ruido)
→ cv2.findContours (RETR_EXTERNAL)
→ Filtrado por área mínima (default: 2000 px²)
→ Filtros geométricos (aspect ratio, solidity, convexity)
→ _measure_contour() → CrackResult
```

Medición por contorno (`_measure_contour`):

- `length_px` = perímetro del convex hull / 2
- `orientation_deg` = ángulo del eje principal via `cv2.fitEllipse()`
- `width_mm` = `area_mm2` / `length_mm` (derivado, no medición directa)
- `pixel_to_mm` = $(\text{pixel_size_um} \times \text{distance_m} \times 1000) / \text{focal_mm}$ — **este es el factor crítico no calibrado**

Clasificación de fisuras (según estándares mineros definidos en código):

Clase	Umbral de ancho	Color en Overlay
fina	< 0.3 mm	Verde teal
media	0.3 – 1.0 mm	Amarillo ámbar
gruesa	> 1.0 mm	Rojo

Filtros geométricos (opcionales, desactivados por defecto):

- `min_aspect_ratio`, `max_aspect_ratio` — descarta contornos cuadrados (no son fisuras)
- `min_solidity` — fisuras tienen solidity baja (son irregulares)
- `min_convexity` — complementa la detección de formas elongadas

Filtro por escala real (`_check_min_size`):

- Si existe `calibration.json` con `pixels_per_mm` > 0: filtra en mm reales (`filter_min_width_mm`, `filter_max_width_mm`)
- Si no hay calibración: filtra en píxeles (`filter_min_width_px`, `filter_min_length_px`)

Carga de calibración (`_load_calibration_scale`): Al iniciar, el detector intenta leer `edge/calibration/calibration.json`. Si `pixels_per_mm` es null (como está actualmente), opera sin escala real. ⚠ **Estado actual:** `pixels_per_mm: null` — **sin calibración activa.**

Pipeline ONNX (`OnnxDetector`)

- Carga cualquier modelo `.onnx` (ej. YOLOv8 exportado)
- Input: (N, 3, H, W) float32 normalizado
- Output esperado: (N, num_detections, 6) — [x1, y1, x2, y2, conf, class]
- Si el modelo no está disponible → **fallback automático** a `OpenCvDetector`
- Estado: **Preparado pero sin modelo entrenado disponible**. Requiere dataset.

4.3 Módulo de Calibración

📁 `edge/edge/calibration/` (5 archivos)

Es el módulo más crítico y el que más trabajo requiere. Está **completamente implementado** pero **no ejecutado** aún.

`calibrate_camera.py` — Calibración intrínseca

- Usa patrón de ajedrez (checkerboard) con `cv2.findChessboardCorners()`
- Calcula: `camera_matrix` (K), `distortion_coefficients` (dist), `reprojection_error`
- Patrón esperado: 9×6 esquinas interiores, cuadrado de 25mm

`detect_aruco_scale.py` — Escala ArUco

- Detecta marcadores ArUco (`cv2.aruco`) en la imagen
- Calcula `pixels_per_mm` midiendo en píxeles el marcador de tamaño conocido
- Estima `distance_m` a la cámara usando la longitud focal

calibration_service.py — Orquestador de calibración

- Coordina los dos pasos anteriores en una sola llamada `run_full_calibration()`
- Persiste todo en `calibration.json` (cargado automáticamente al iniciar el detector)
- También ofrece `quick_scale_from_aruco()` para actualizar solo la escala

Estado del `calibration.json`:

```
{
  "calibration_date": null,
  "camera_matrix": null,
  "fx_px": null, "fy_px": null, "cx_px": null, "cy_px": null,
  "reprojection_error": null,
  "pixels_per_mm": null,
  "calibrated": false,
  "scale_detected": false
}
```

⚠ BRECHA CRÍTICA: Todos los campos son `null`. Las mediciones en mm que el sistema calcula y publica actualmente **son estimaciones no validadas** basadas en los parámetros físicos por defecto del `.env`.

Parámetros por defecto activos en producción:

```
FOCAL_LENGTH_MM    = 50.0  mm ← valor genérico, no medido
SENSOR_DISTANCE_M  = 10.0  m  ← distancia al talud no verificada
SENSOR_PIXEL_UM    = 3.0   μm ← tamaño de pixel asumido
```

La fórmula activa: `pixel_to_mm = (3.0/1e6 × 10000) / 50.0 = 0.0006 mm/px`

4.4 Estimación de Profundidad — MiDaS

 `edge/edge/depth/midas_depth.py` (218 líneas)

Implementa **MiDaS v3.1 Small** (modelo de IA de Intel) para estimación de profundidad monocular en CPU.

Flujo técnico:

1. `torch.hub.load("intel-isl/MiDaS", "MiDaS_small")` — cargado lazy en primer frame
2. Conversión BGR→RGB, aplicar transform de MiDaS
3. `torch.no_grad()` — inferencia en CPU
4. Resize al tamaño original del frame
5. **Normalización crítica** (comentada en el código):
 - MiDaS retorna **disparidad inversa** (mayor = más cercano)
 - Se invierte para obtener profundidad (mayor = más lejano)
 - Clipping percentil 5–95 para suprimir outliers de paredes planas

- Escalado al rango métrico [2.0m – 5.0m] (configurado para taludes)

Limitación documentada en el propio código:

"Para una pared plana, MiDaS disparity es casi uniforme (± 0.03). La normalización completa [0,1] amplificaría estas variaciones minúsculas a 4.7m $\rightarrow$ forma de cono artificial."

⚠ IMPORTANTE: MiDaS es **profundidad relativa monocular**, no LIDAR ni estéreo. El rango 2–5m es un escalado matemático heurístico, **no una medición física absoluta**. Para un gemelo digital métrico real, se necesita cámara estéreo o LIDAR.

Throttling de rendimiento: El pipeline 3D se ejecuta a `DEPTH_FPS=5` fps (no en cada frame), calculando el intervalo dinámicamente: `depth_frame_interval = fps / depth_fps`.

4.5 Generación de Nube de Puntos y Publicación 3D

`edge/edge/pointcloud/generator.py` (no leído completo) + `mqtt/publisher.py`

Proceso de publicación (`publish_alert_3d`):

1. Submuestra la nube de puntos a máximo `pointcloud_max_points` (default: 10.000 pts)
2. Para cada fisura detectada, proyecta sus coordenadas 2D al espacio 3D:

```
x3d = (cx2d - cx) * depth / fx
y3d = (cy2d - cy) * depth / fy
z3d = depth # del mapa de MiDaS
```

3. Los intrínsecos usados actualmente son **fijos y mockeados**:

```
fx=1408.0, fy=1408.0, cx=640.0, cy=360.0
```

⚠ Estos valores deberían provenir de `calibration.json` tras calibrar la cámara.

Payload MQTT al topic `mineria/talud/alertas`:

```
{
  "device_id": "argos-edge-01",
  "timestamp": "2026-06-07T18:00:00",
  "point_cloud": [[x,y,z,r,g,b], ...], // hasta 10.000 puntos
  "cracks": [
    {
      "x": 100, "y": 200, "w": 50, "h": 30,
      "x3d": 0.12, "y3d": -0.08, "z3d": 3.45,
      "classification": "media", "roi_id": "CRK-A3B2"
    }
  ],
  "point_count": 8500
}
```

4.6 Transmisión de Video — WebRTC y MJPEG

📄 [edge/edge/webrtc/](#)

Dos servidores corren en **threads daemon** paralelos al loop principal:

Servidor	Puerto	Protocolo	Estado
MJPEG	:8082/stream	HTTP multipart	Activo por defecto
WebRTC Signaling	:8081	WebSocket + aiortc	Activo pero experimental

El frontend ofrece un toggle **MJPEG / WebRTC** en la UI. MJPEG es el modo recomendado porque "funciona siempre" sin negociación ICE.

El frame enviado al stream es **proc_frame** (ya procesado y con detecciones calculadas).

4.7 Backend .NET — API REST

📄 [backend/src/ArgosSlope.Api/](#)

Modelo de datos en PostgreSQL (evidenciado por clases C#):

```

fisura
├ id, roi_id
├ fecha_deteccion
├ largo_mm, ancho_mm, area_mm2
├ orientacion, tipo (fina/media/gruesa)
├ coordenadas (JSON: {x,y,w,h})
├ imagen_original, imagen_segmentada (paths)
├ → medicion_diaria (1:N)
├ → alerta (1:N)

medicion_diaria
├ fisura_id (FK)
├ fecha
├ largo_mm, ancho_mm, area_mm2
├ delta_porcentaje (Δ% respecto a medición anterior)
├ es_critica (bool – supera umbral)

alerta
├ fisura_id (FK)
├ fecha, tipo, mensaje
├ umbral_superado, valor_actual
├ reconocida (bool)
```

Endpoints disponibles (**FisurasController**):

Método	Ruta	Función
GET	/api/fisuras	Lista todas las fisuras detectadas
GET	/api/fisuras/{id}	Detalle completo + mediciones + alertas
GET	/api/fisuras/{id}/mediciones?dias=N	Serie temporal de mediciones

El esquema de `medicion_diaria` con `delta_porcentaje` y `es_critica` es la base del **Risk Engine y comparación temporal**. La lógica de cálculo del delta y la evaluación del umbral está diseñada pero pendiente de la ingesta de datos reales calibrados.

4.8 Frontend Next.js — Visualización 3D y Dashboard

`frontend/app/` y `frontend/components/`

Componentes críticos implementados:

NubePuntos3D.tsx — Render 3D con React Three Fiber:

- Recibe `points: [x,y,z,r,g,b][]` desde MQTT
- Construye `THREE.BufferGeometry` con `Float32BufferAttribute`
- Renderiza fisuras como **esferas rojas** (`#ff4444`) en posición 3D exacta
- Corrección de coordenadas documentada en código: `scale=[-1, -1, 1]` para voltear eje Y (sistema OpenCV → Three.js)
- `OrbitControls` con damping, autoRotate y límites de zoom
- Vacío → muestra `EmptyState` esperando datos del edge

FisuraOverlay.tsx — Overlay Canvas sobre video:

- Se conecta al broker MQTT via WebSocket (`mqtt.connect()`)
- Suscribe al topic `argos/+/fisura`
- Dibuja `bounding boxes` sobre canvas transparente superpuesto al video
- Colores por clasificación: Verde (fina) / Ámbar (media) / Rojo (gruesa)
- Muestra `roi_id`, clasificación y `length_mm` formateado
- Barra de confianza dibujada debajo del bounding box
- **Auto-limpieza:** Fisuras con más de 5 segundos sin actualización se eliminan del canvas

VideoEnVivo.tsx — Cliente WebRTC nativo:

- Conexión WebSocket al servidor de señalización en `:8081`
- Negociación SDP completa (offer/answer/ICE candidates)
- `RTCPeerConnection` con video track del edge

CamaraMJPEG.tsx — Fallback MJPEG:

- `` → actualización automática del browser
- Modo recomendado en UI (más simple y confiable)

Páginas del sistema:

Ruta	Componente	Función
/monitoreo	VideoEnVivo + FisuraOverlay	Video en vivo con detecciones
/visualizacion	NubePuntos3D + VisorModelo3D	Gemelo digital 3D
/historial	TablaHistorial + GraficoDeformacion	Serie temporal de fisuras
/alertas	PanelAlertas	Lista de alertas activas
/configuracion	FormularioConfig	Parámetros de conexión

5. Estado Consolidado de Componentes

Componente	Archivo(s) clave	Estado real	Detalle
Orquestador Edge	main.py	 Completado	506 líneas, producción-ready en laptop
Captura de Cámara	main.py:open_camera()	 Completado	DirectShow (Windows) / V4L2 (Linux), FPS configurable
Preprocesamiento	preprocessing/preprocessor.py	 Completado	undistort + CLAHE por pipeline
Detector OpenCV	detector/fisura_detector.py	 Completado	Umbral adaptativo + morfología + filtros geométricos
Detector ONNX/ML	detector/fisura_detector.py	 Preparado	Infraestructura lista, sin modelo entrenado
Calibración intrínseca	calibration/calibrate_camera.py	 Implementado, sin ejecutar	Código listo, calibration.json vacío
Escala ArUco	calibration/detect_aruco_scale.py	 Implementado, sin ejecutar	Requiere imagen con marcador
Estimación Profundidad	depth/midas_depth.py	 Operativo	MiDaS Small CPU, normalización percentil implementada

Componente	Archivo(s) clave	Estado real	Detalle
Nube de Puntos 3D	pointcloud/generator.py	✓ Operativo	Genera <code>[x,y,z,r,g,b]</code> , subsampleado a 10K pts
Proyección 3D de fisuras	mqtt/publisher.py	● Funcional con intrínsecos fijos	Usa <code>fx=1408</code> hardcodeado, no calibrado
Transmisión MJPEG	webrtc/mjpeg_server.py	✓ Operativo	:8082/stream
Transmisión WebRTC	webrtc/signaling_server.py	● Experimental	Funciona, pero MJPEG es recomendado
MQTT Publisher	mqtt/publisher.py	✓ Completado	487 líneas, TLS, QoS, reconexión exponencial
Simulador	simulator/crack_simulator.py	✓ Completado	Generador de eventos sintéticos, desactivado por defecto
Backend API .NET	Controllers/, Models/	✓ Estructura completa	4 controllers, modelos bien tipados, ORM
Modelo de BD	Fisura.cs, MedicionDiaria.cs, Alerta.cs	✓ Diseñado	Esquema relacional con delta%, critica, historial
Frontend Video	CamaraMJPEG.tsx, VideoEnVivo.tsx	✓ Completado	Soporte MJPEG + WebRTC con toggle
Overlay Fisuras	FisuraOverlay.tsx	✓ Completado	Canvas MQTT en tiempo real, auto-limpieza 5s
Visualización 3D	NubePuntos3D.tsx	✓ Completado	Three.js/R3F, esferas de fisuras, orbit controls
Dashboard/Historial	PanelAlertas.tsx, TablaHistorial.js	● En desarrollo	Componentes listos, falta datos reales
Risk Engine	—	✗ Pendiente	Lógica de umbral por definir, modelo <code>es_critica</code> listo en BD

Componente	Archivo(s) clave	Estado real	Detalle
Comparación temporal	—	✗ Pendiente	Infraestructura BD lista (delta_porcentaje), sin algoritmo
Raspberry Pi Edge	main.py (compatible)	● Planificado	Código 100% compatible con ARM, sin hardware configurado
HD Capture (meshing)	capture_hd/manager.py	● Implementado	Open3D meshing on-demand vía MQTT trigger
WebXR / VR	—	✗ No iniciado	Dependiente del gemelo digital métrico real

6. Brechas Críticas Confirmadas por Código

● Brecha 1: Calibración no ejecutada (BLOQUEANTE)

Evidencia: **calibration.json** tiene "**pixels_per_mm**": null, "**calibrated**": false.

Impacto: Los valores de **length_mm**, **width_mm** y **area_mm2** publicados vía MQTT y almacenados en PostgreSQL son estimaciones con parámetros genéricos (**focal=50mm**, **dist=10m**, **pixel=3μm**). No son representativos de la cámara real del sistema.

Solución técnica: Ejecutar **CalibrationService.run_full_calibration()** con:

1. 10–20 fotos de tablero de ajedrez (9×6, cuadros de 25mm)
2. Una imagen con marcador ArUco de 100mm de tamaño conocido

● Brecha 2: Intrínsecos 3D hardcodedos

Evidencia: En **publisher.py**, líneas 369–372:

```
fx = intrinsics.get("fx", 1408.0) if intrinsics else 1408.0
```

Y en **main.py**, línea 438:

```
intrinsics = {"fx": 1408.0, "fy": 1408.0, "cx": 640.0, "cy": 360.0}
```

Impacto: Las posiciones 3D de las fisuras (**x3d**, **y3d**, **z3d**) en la nube de puntos no son métricamente correctas. La visualización 3D es topológica, no topográfica.

Solución: Tras calibrar, cargar `fx_px` y `cx_px` del `calibration.json` y pasarlos al publisher.

● Brecha 3: MiDaS genera profundidad relativa, no absoluta

Evidencia: Código en `midas_depth.py` líneas 196–198:

```
DEPTH_MIN_M = 2.0
DEPTH_MAX_M = 5.0
depth = depth * (DEPTH_MAX_M - DEPTH_MIN_M) + DEPTH_MIN_M
```

Impacto: El rango 2–5m es una suposición del programador para taludes. No hay sensor de profundidad físico. Para medición real se necesita cámara estéreo o LIDAR.

● Brecha 4: Risk Engine sin lógica de umbral real

Evidencia: El modelo `MedicionDiaria.cs` tiene `EsCritica` y `DeltaPorcentaje`, pero no existe código de evaluación del umbral en el backend (`Services/`).

● Brecha 5: Comparación temporal sin algoritmo de alineación

Evidencia: La BD puede almacenar mediciones diarias con `DeltaPorcentaje`, pero no hay implementación de alineación de imágenes (homografía) para comparar frames en el tiempo.

7. Avance Técnico para Informe Formal

A la fecha de este informe, el proyecto ARGOS SLOPE 4.0 ha completado el desarrollo del **núcleo funcional** de su arquitectura distribuida. El módulo de procesamiento Edge ejecuta en producción un pipeline completo que comprende: adquisición de video desde cámara USB, preprocesamiento de imagen (corrección CLAHE y undistort), detección de fisuras mediante análisis de contornos adaptativos con OpenCV, estimación de profundidad monocular via red neuronal MiDaS v3.1, generación de nubes de puntos tridimensionales en formato `[x,y,z,r,g,b]`, y transmisión concurrente del video procesado mediante los protocolos MJPEG y WebRTC. La comunicación entre el módulo Edge y los servicios de backend se realiza íntegramente vía mensajería MQTT (QoS 1), con esquemas de payload JSON bien definidos y reconexión automática con backoff exponencial.

El subsistema de visualización, implementado en Next.js con React Three Fiber, recibe y renderiza en tiempo real la nube de puntos 3D del talud, superpone las fisuras detectadas como marcadores tridimensionales y presenta bounding boxes de detección sobre el video en vivo mediante un canvas transparente reactivo. El backend en .NET expone una API REST con un modelo relacional en PostgreSQL que contempla el historial de fisuras, mediciones diarias y alertas con su clasificación de criticidad.

La etapa inmediatamente prioritaria para escalar este prototipo a un sistema de monitoreo geotécnico confiable es la ejecución de la **calibración óptica de la cámara** mediante tablero de ajedrez y marcadores ArUco, procedimiento que generará los parámetros intrínsecos reales y el factor de conversión px/mm que actualmente el sistema asume con valores genéricos.

8. Qué se puede presentar al equipo ahora

● Presentable con confianza total (evidenciado y operativo)

- **Pipeline completo Edge en tiempo real:** captura → detección → publicación MQTT
- **Transmisión de video en vivo** con bounding boxes de fisuras superpuestos (MJPEG)
- **Nube de puntos 3D** recibida y visualizada en navegador via React Three Fiber
- **Clasificación de fisuras** en fina/media/gruesa con colores diferenciados
- **Arquitectura MQTT** con topics definidos y payload estructurado
- **Modelo de datos PostgreSQL** completo: fisura, medición diaria, alerta
- **Dashboard multi-sección:** monitoreo, visualización 3D, historial, alertas

● Presentable con aclaración (funcional pero con limitaciones conocidas)

- **Mediciones en mm** → *Aclaración: calculadas con parámetros físicos estimados, pendiente calibración con cámara real*
- **Posiciones 3D de fisuras** → *Aclaración: usando intrínsecos fijos, no calibrados*
- **Profundidad MiDaS** → *Aclaración: IA monocular escalada a 2–5m, no es LIDAR*
- **Detector ONNX** → *Aclaración: infraestructura lista, sin modelo entrenado aún*

● No prometer en esta etapa

- Precisión milimétrica certificada para monitoreo geotécnico real
- Comparación temporal robusta de crecimiento de fisuras
- Risk Engine con umbrales geotécnicos validados
- Visualización WebXR / Realidad Virtual funcional
- Detección sin falsos positivos en campo (rocas, vegetación, luz solar)

9. Roadmap con Estado Actual

Fase	Objetivo	Estado	Criterio de aceptación
F1 — Captura local	Cámara estable en laptop	✓ Completado	Video continuo, FPS configurables
F2 — Pipeline detección	OpenCV → MQTT	✓ Completado	<code>CrackResult</code> con clasificación publicado
F3 — Calibración	Factor px/mm real	● Pendiente	<code>calibration.json</code> con <code>pixels_per_mm > 0</code>
F4 — Backend e historial	Persistencia real	● Parcial	API operativa, falta ingestión de datos reales
F5 — Comparación temporal	Detección de crecimiento	✗ Pendiente	<code>delta_porcentaje</code> calculado automáticamente
F6 — Risk Engine	Alertas geotécnicas	✗ Pendiente	Umbrales configurables, alertas con <code>is_critical=true</code>
F7 — Gemelo digital 3D	Fisuras proyectadas con métrica	● Parcial	Visualización correcta tras calibración

Fase	Objetivo	Estado	Criterio de aceptación
F8 — Raspberry Pi edge	Captura remota	 Planificado	Edge corre en ARM, datos llegan al servidor
F9 — WebXR	Navegación inmersiva	 No iniciado	Dependiente de F3–F7

10. Tareas Técnicas Prioritarias (Próximas 2 semanas)

Semana 1 — Cerrar la brecha de calibración

1. **Imprimir tablero de ajedrez** 9×6, cuadros de 25mm exactos
2. **Tomar 15–20 fotos** del tablero con la misma cámara USB, distintos ángulos
3. **Imprimir marcador ArUco** de 100mm, colocarlo en la escena del talud
4. **Ejecutar** `CalibrationService.run_full_calibration()` y verificar que `calibration.json` quede con `calibrated: true`
5. **Conectar intrínsecos** de `calibration.json` al `publisher.py` (reemplazar `fx=1408.0` por los reales)
6. **Reiniciar el edge** y verificar que `length_mm` cambie respecto al valor anterior

Semana 2 — Datos reales al backend y validación

7. **Habilitar escritura a PostgreSQL** desde el worker MQTT del backend
8. **Probar flujo completo**: fisura detectada → MQTT → backend → BD → frontend historial
9. **Implementar cálculo de delta_porcentaje** al guardar nueva medición diaria
10. **Validar con imágenes reales del talud** que el detector no genere exceso de falsos positivos
11. **Documentar parámetros ajustados** de `ADAPTIVE_BLOCK`, `ADAPTIVE_C` y `MIN_CONTOUR_AREA` para el entorno real

11. Riesgos Técnicos

Riesgo	Probabilidad	Impacto	Mitigación
Calibración difícil por condiciones de luz en talud	Alta	Alto	Realizar calibración indoor; usar solo escala ArUco en campo
MiDaS no distingue profundidad real en roca uniforme	Alta	Medio	Documentar limitación; evaluar cámara estéreo para fase futura
Alto CPU por MiDaS + WebRTC en laptop simultáneo	Media	Medio	<code>DEPTH_FPS=2</code> o <code>DEPTH_ENABLED=false</code> si hay throttling
Falsos positivos en campo (sombras, polvo, vegetación)	Alta	Alto	Ajustar <code>ADAPTIVE_BLOCK</code> , <code>MIN_CONTOUR_AREA</code> y activar filtros geométricos
<code>SIMULATOR_ENABLED=true</code> accidentalmente en producción	Baja	Alto	Verificar <code>.env</code> antes de deploy; agregar log de advertencia visible

Riesgo	Probabilidad	Impacto	Mitigación
Raspberry Pi sin microSD retrasa pruebas en campo	Actual	Medio	Continuar validando con laptop; priorizar calibración

12. Conclusión

El proyecto ARGOS SLOPE 4.0 tiene un **núcleo técnico construido y en ejecución**. La arquitectura distribuida (Edge Python → MQTT → Backend .NET → PostgreSQL → Frontend Next.js → Visualización 3D) está operativa y demuestra el flujo completo del sistema. El detector de fisuras clasifica correctamente por severidad, la nube de puntos se visualiza en 3D en tiempo real, y el modelo de datos soporta el historial evolutivo que el Risk Engine necesitará.

La prioridad absoluta es ejecutar la calibración óptica de la cámara. Sin ella, el sistema funciona como una demostración visual convincente, pero sus mediciones en milímetros carecen de validez geotécnica. Una vez calibrado, el sistema estará listo para producir datos reales que alimenten el historial, el motor de riesgo y el gemelo digital métrico.

Pitch de 1 Minuto (Exposición oral)

"Buenas, equipo. El core de ARGOS SLOPE 4.0 ya está construido y funcionando. Tenemos un pipeline completo: la cámara captura video, OpenCV detecta fisuras en tiempo real, una IA calcula la profundidad del talud, y todo eso se transmite en vivo al dashboard con una nube de puntos 3D navegable en el navegador.

Lo que ven en pantalla es real: el video viene del edge, los bounding boxes de fisuras vienen de MQTT, y la visualización 3D se actualiza automáticamente.

Ahora bien, siendo honestos: las medidas en milímetros que muestra el sistema hoy están calculadas con parámetros genéricos, porque todavía no hemos calibrado la cámara. Esa es nuestra tarea número uno de esta semana: imprimir un tablero de ajedrez, tomar las fotos de calibración, y dejar el sistema midiendo con precisión real. Con eso resuelto, el resto del trabajo —historial, alertas, risk engine— ya tiene la infraestructura lista para recibir los datos."

Documento generado con análisis directo del código fuente del repositorio. Módulos revisados: `edge/main.py`, `detector/fisura_detector.py`, `calibration/calibration_service.py`, `calibration/calibration.json`, `depth/midas_depth.py`, `mqtt/publisher.py`, `config.py`, `backend/Models/*.cs`, `backend/Controllers/FisurasController.cs`, `frontend/components/NubePuntos3D.tsx`, `frontend/components/FisuraOverlay.tsx`, `frontend/app/monitoreo/page.js`